תרגיל 2 : פונקציות

תאריך פרסום:	25.10.2023
תאריך הגשה:	יפורסם בהמשך
מתרגלת אחראית:	אסראא נסאסרה
משקל תרגיל:	2 נקודות
מטרות העבודה:	שימוש בפונקציות ועבודה עם מחרוזות

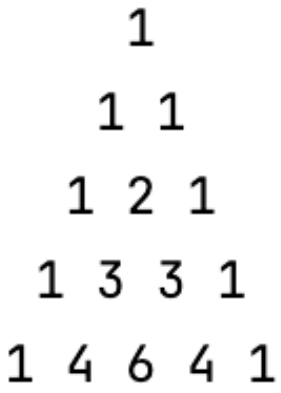
דגשים לתרגיל 2

- במימוש תרגיל זה עליכם להשתמש אך ורק בפקודות הבאות, כלומר אין להשתמש בפקודות אחרות, אפילו אם הן נלמדו בשיעור.
  - תנאים: if, elif, else, in וכל פעולות ההשוואה שנלמדו בכיתה.
  - לולאות: for, while, continue, break, in, range, וכו'.
  - ניתן להשתמש במשתנים מספריים, בוליאניים, תווים, מחרוזות ורשימות.
    אין להשתמש בסוגי משתנים אחרים (כגון מילונים, tuple, סטים וכו').
  - הפונקציות האריתמטיות (+, -, ** וכו') והלוגיות (and, or וכו') של פייתון.
  - מתוך הפונקציות המובנות של פייתון ניתן להשתמש רק באלו:
    len, chr, str, ord, int, bool, float, list, pow, type, range, min, max, print, input.
  - כל השיטות של String (אלא אם כן נאמר אחרת).
- ניתן להוסיף פונקציות עזר.
- חובה לכתוב docstring (כפי שלמדתם בתרגול) לכל פונקציה.
- העבודה כוללת 3 קבצים: קובץ ייעודי לכל שאלה (question) וקובץ output.txt המכיל את הפלטים של דוגמאות ההרצה עבור שאלה 2- סעיף ג'. השתמשו בו על מנת להשוות את הפלט שלכם לתוכן של קובץ זה.

בהצלחה!

שאלה 1 :

משולש פסקל הוא משולש המורכב ממספרים לפי התיאור הבא. הקדקוד העליון של משולש פסקל הוא המספר 1, וכל מספר אחר במשולש הוא סכום שני המספרים שנמצאים מעליו במשולש (כאשר בקצוות - המספר החסר לחיבור יהיה 0). משולש פסקל מוגדר לפי *הגובה* - מספר השורות במשולש. לדוגמה, להלן משולש פסקל בגובה של 5 שורות :



סעיף א :

ממשו את הפונקצייה בשם :

```
def generate_pascals_triangle(rows)
```

הפונקציה מקבלת כקלט rows מספר שלם וחיובי, המייצג את גובה משולש הפסקל. הפונקציה מחזירה משולש פסקל כרשימה של רשימות כאשר כל איבר הוא רשימת מספרים שלמים המייצגים שורה במשולש. הניחו כי בשאלה זו הקלט תקין ו-rows הוא מספר שלם וחיובי.

דוגמאות ריצה :

```
>>> generate_pascals_triangle(5)
[[1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1]]
>>> generate_pascals_triangle(3)
[[1], [1, 1], [1, 2, 1]]
>>> generate_pascals_triangle(7)
[[1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1], [1, 5, 10, 10, 5, 1], [1, 6, 15, 20, 15, 6, 1]]
>>> generate_pascals_triangle(4)
[[1], [1, 1], [1, 2, 1], [1, 3, 3, 1]]
>>> generate_pascals_triangle(1)
[[1]]
```

סעיף ב:

ממשו את הפונקציה:

def print_pascals_triangle(triangle):

הפונקציה מקבלת משולש פסקל (בפורמט הפלט של סעיף א'), ומדפיסה את המשולש למסך לפי הפורמט בדוגמא המופיעה טרם סעיף א' (משולש פסקל בגובה 5 שורות).

רמז:

אתם רשאים להשתמש בשיטה center המובנית של מחרוזות בפייתון. השיטה ממרכזת את התווים של מחרוזת ע"י ריפוד המחרוזת ברווחים משני צדדיה בהתאם לאורך רצוי. למשל, הערכים במשולש הפסקל בדוגמה למעלה מרופדים ברווחים לפי אורך המחרוזת בשורה האחרונה.

דוגמת ריצה:

```
>>> pascals_triangle = generate_pascals_triangle(5)
... print_pascals_triangle(pascals_triangle)
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```

שאלה 2 :

"איש תלוי" הוא משחק לשני שחקנים או יותר. מטרת המשחק היא לנחש נכון את האותיות המרכיבות מילה שנבחרה על ידי אחד השחקנים, בהינתן מספר ניסיונות מותר שנבחר מראש. בשאלה זו תממשו מערכת משחק המאפשרת לשחקן אנושי להתחרות נגד המחשב, כאשר השחקן מנסה לנחש את המילה שהמחשב בחר.

<u>חוקי המשחק :</u>

א. מילה חוקית במשחק היא אוסף של אותיות אלפביתיות קטנות בלבד.

ב. המחשב מקבל אוסף של מילים חוקיות שונות (לכל הפחות שתיים) המופרדות ע"י התו $. אוסף מילים זה מהווה מאגר ממנו המחשב בוחר את המילה שיש לנחש.

ג. מהלך המשחק :

i. השחקן בוחר את מספר הניסיונות המותר. כלומר, אם השחקן לא יצליח לנחש את כל האותיות המרכיבות את המילה תוך מספר ניסיונות זה, הוא יפסיד במשחק.

ii. בכל סיבוב, השחקן מנחש אות. אם האות נמצאת במילה, המחשב מדפיס את המילה עם כל האותיות שנוחשו עד כה במקומן הנכון במילה (תמונת מצב). אחרת, המחשב ידפיס הודעה מתאימה ומספר הניסיונות המותר קטן באחד.

iii. ניחוש חוזר של אות שנמצאת במילה אינו משנה את מספר הניסיונות המותר. אם המילה שיש לנחש מכילה אות פעמיים או יותר, ניחוש האות פעם אחת שקול לניחוש כל מופעיה במילה.

iv. המשחק מסתיים כאשר המשתמש מנחש את כל האותיות המרכיבות את המילה, או כשנכשל לנחש את כל אותיות המילה במספר הניסיונות המותר.

דוגמא למהלך משחק אפשרי :

```
>>> play_hangman("holiday$program$process$arrange$example")
Welcome to Hangman Game!
Enter number of attempts: >? 5
The word has 7 letters. You have 5 attempts.
_ _ _ _ _ _ _
Guess a letter: >? p
Wrong guess! Attempts remaining: 4
_ _ _ _ _ _ _
Guess a letter: >? h
h _ _ _ _ _ _
Guess a letter: >? o
h o _ _ _ _ _
Guess a letter: >? d
h o _ _ d _ _
Guess a letter: >? a
h o _ _ d a _
Guess a letter: >? l
h o l _ d a _
Guess a letter: >? i
h o l i d a _
Guess a letter: >? y
Congratulations! You guessed the word: holiday
```

סעיף א:

ממשו את הפונקציה:

def split_words(game_words)

המקבלת כקלט game_words, מחרוזת המורכבת ממילים שונות ומופרדות אחת מהשנייה ע"י התו "$". הפונקציה מחזירה את רשימת המילים המתאימה למחרוזת הקלט. במימושכם אין להשתמש בשיטה split של מחרוזות.

דוגמאות ריצה:

```
>>> ans = split_words("hello$world$great$years$before$winter")
... print(ans)
['hello', 'world', 'great', 'years', 'before', 'winter']
>>> print(type(ans))
<class 'list'>
>>> print(type(ans[0]))
<class 'str'>
```

```
>>> ans = split_words("fly$event$custom$arrange$game$floor$sky$range")
... print(ans)
['fly', 'event', 'custom', 'arrange', 'game', 'floor', 'sky', 'range']
```

סעיף ב:

ממשו את הפונקציה:

def get_guess_result (secret_word, guess_letter, currunt_word):

הפונקציה מתארת מהלך סיבוב במשחק (ראו סעיף ג'ii בחוקי המשחק בעמוד הקודם), המקבלת את 3 הקלטים הבאים:

- secret_word, מחרוזת המייצגת מילה חוקית שצריך לנחש.
- guess_letter, מחרוזת המכילה תו אחד (אות).
- currunt_word, רשימה המכילה מחרוזות של תו אחד כמספר האותיות של מילת הניחוש. הרשימה מייצגת תמונת מצב לניחוש. כל תו ברשימה יכול להיות אות אשר נוחשה נכון או מקף תחתון "_" לאותיות שעדיין לא נוחשו.

הפונקציה מחזירה רשימה של מחרוזות, המהווה את תמונת המצב לאחר הניחוש הנוכחי. אם האות שנוחשה נמצאת במילה, יש לשנות את תמונת המצב ע"י החלפת כל מקומות מופעיה ברשימה מ-"_" לאות עצמה. תמונת המצב לא משתנה אם האות שנוחשה לא נמצאת במילה.

דוגמאות ריצה:

```
>>> curr_word = get_guess_result("the","a", ["_","_","_"])
... print(curr_word)
... curr_word = get_guess_result("the","e", curr_word)
... print(curr_word)
... curr_word = get_guess_result("the","t", curr_word)
... print(curr_word)
... curr_word = get_guess_result("the","h", curr_word)
... print(curr_word)
['_', '_', '_']
['_', '_', 'e']
['t', '_', 'e']
['t', 'h', 'e']
```

```
>>> curr_word = get_guess_result("bold","a", ["b","_","l","_"])
... print(curr_word)
... curr_word = get_guess_result("bold","d", curr_word)
... print(curr_word)
['b', '_', 'l', '_']
['b', '_', 'l', 'd']
```

סעיף ג:

כפי שהוזכר בתחילת השאלה, במהלך משחק של "איש תלוי" המחשב בוחר מילה בצורה אקראית מתוך מאגר מילים המקודד ע"י מחרוזת המורכת מאוסף של מילים שונות ומופרדות אחת מהשנייה ע"י התו "$". לאחר מכן, על המשתמש לבחור את מספר הניסיונות המותר לו במשחק ואז מתחיל המשתמש לשחק את המשחק עד לסיומו כך שסיום המשחק מוגדר ע"י ניחוש מוצלח של המילה או תום מספר הניסיונות המותר.

בסעיף זה תממשו את פונקציית המשחק play_hangman, המבצעת מהלך משחק כזה ועושה שימוש בפונקציות שמימשתם בסעיפים הקודמים.

def play_hangman(game_words):

הפונקציה מקבלת כקלט game_words, מחרוזת המורכבת ממילים שונות ומופרדות אחת מהשנייה ע"י התו "$". הפונקציה מתחילה את מהלך המשחק ע"י בחירת מילה לניחוש בצורה אקראית מתוך המאגר המקודד ע"י מחרוזת הקלט ולאחר מכן מאפשרת למשתמש לנחש את המילה לפי מהלך משחק (יוסבר בהמשך).

מהלך המשחק:

בתחילת מהלך המשחק, המחשב בוחר את מילת הניחוש בצורה אקראית מתוך המאגר המקודד ע"י מחרוזת הקלט game_words. לשם בחירת מילה בצורה אקראית מתוך המאגר, נתון לכם מימוש פונקציית העזר choose_secret_word(words_list) המקבלת כקלט רשימת מחרוזות המייצגות מילים שונות (בהתאם לפורמט הפלט של הפונקציה שמימשתם בסעיף א). הפונקציה choose_secret_word מגרילה מילה אחת בצורה אקראית מתוך רשימת מחרוזות הקלט ומחזירה אותה. השתמשו בפונקציית העזר על מנת לאתחל את המילה אותה המשתמש יצטרך לנחש.

לאחר מכן, תודפס ההודעה הבאה:

"Welcome to Hangman Game!"

בשלב הבא עליכם לקבל קלט מהמשתמש שיבחר את מספר הניסיונות המותר שיהיו לו במהלך המשחק בצירוף ההודעה הבאה:

"Enter number of attempts: "

לאחר מכן, עליכם להדפיס הודעה למשתמש המכריזה על תחילת המשחק. על ההודעה להיות בפורמט הבא: לדוגמה עבור מילה בת 7 אותיות (למשל holiday) שיש לנחש ומספר ניסיונות מותר של 5, יש להדפיס:

"The word has 7 letters. You have 5 attempts.

_ _ _ _ _ _ _"

בכל נסיון עליכם לקבל מהמשתמש את הניחוש שלו:

"Guess a letter: "

אם הניחוש נכון תודפס למשתמש תמונת המצב של המשחק, עם האותיות שהוא ניחש נכון עד כה במקומן במילה, והאותיות החסרות יוצגו ע"י קו תחתון '_'. לדוגמה בהינתן המילה holiday וניחוש נכון ראשון של האות "i" תודפס ההודעה הבאה:

"_ _ _ i _ _ _"

במידה והמשתמש ניחש אות שאינה שייכת למילה, מספר הניסיונות המותר שלו יפחת ב-1 וההודעה הבאה תודפס למסך:

"Wrong guess! Attempts remaining: 4

_ _ _ i _ _ _"

במידה והמשתמש ניחש את אותה האות פעמיים (בין אם האות נמצאת במילה או לא), מספר הניסיונות שלו לא ישתנה והמחשב ידפיס את ההודעה הבא:

"You already guessed that letter."

במידה והמשתמש הצליח לנחש את כל אותיות המילה לפני תום הניסיונות המותר, תודפס ההודעה הבאה:

"Congratulations! You guessed the word: holiday"

במידה ונגמרו מספר הניסיונות המותר והמשתמש עדיין לא ניחש נכון את כל האותיות תודפס למסך ההודעה הבאה:

"Wrong guess! Attempts remaining: 0

Sorry, you've run out of attempts. The word was: holiday"

על המשחק לרוץ עד לסיום (ניחוש מוצלח של המילה או תום מספר הניסיונות המותר).

דוגמאות ריצה:

קיים לרשותכם הקובץ outputs.txt שמכיל את דוגמאות ההרצה הבאות, השתמשו בו להשוות את הפלטים שלכם לפלט הנדרש.

דוגמה 1:

```
>>> play_hangman("holiday$program$process$arrange$example")
Welcome to Hangman Game!
Enter number of attempts: >? 4
The word has 7 letters. You have 4 attempts.
_ _ _ _ _ _ _
Guess a letter: >? g
_ _ _ _ _ g _
Guess a letter: >? p
Wrong guess! Attempts remaining: 3
_ _ _ _ _ g _
Guess a letter: >? a
a _ _ a _ g _
Guess a letter: >? o
Wrong guess! Attempts remaining: 2
a _ _ a _ g _
Guess a letter: >? r
a r r a _ g _
Guess a letter: >? n
a r r a n g _
Guess a letter: >? e
Congratulations! You guessed the word: arrange
```

דוגמה 2:

```
>>> play_hangman("exam$program$question$floor$summer")
Welcome to Hangman Game!
Enter number of attempts: >? 2
The word has 4 letters. You have 2 attempts.
_ _ _ _
Guess a letter: >? t
Wrong guess! Attempts remaining: 1
_ _ _ _
Guess a letter: >? r
Wrong guess! Attempts remaining: 0
Sorry, you've run out of attempts. The word was: exam
```

דוגמה 3 :

```
>>> play_hangman("fly$event$custom$arrange$game$floor$sky$range")
Welcome to Hangman Game!
Enter number of attempts: >? 5
The word has 5 letters. You have 5 attempts.
_ _ _ _ _
Guess a letter: >? t
Wrong guess! Attempts remaining: 4
_ _ _ _ _
Guess a letter: >? t
You already guessed that letter.
_ _ _ _ _
Guess a letter: >? a
_ a _ _ _
Guess a letter: >? n
_ a n _ _
Guess a letter: >? n
You already guessed that letter.
_ a n _ _
Guess a letter: >? f
Wrong guess! Attempts remaining: 3
_ a n _ _
Guess a letter: >? r
r a n _ _
Guess a letter: >? g
r a n g _
Guess a letter: >? e
Congratulations! You guessed the word: range
```